6.Implementation of a Simple Artificial Neural Network (ANN) Using the Iris Dataset

Theory

An Artificial Neural Network (ANN) is a machine learning model inspired by the human brain. It consists of interconnected neurons that process information and learn patterns from data. ANNs are widely used for classification, regression, image recognition, speech recognition, and many other AI applications.

A basic ANN consists of three layers:

- **Input Layer:** Receives the input features.
- **Hidden Layer:** Performs computations using weights, biases, and activation functions.
- **Output Layer:** Produces the final prediction.

Iris Dataset

The Iris dataset is one of the most popular datasets used for machine learning.

Features

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

Target Classes

- Iris Setosa
- Iris Versicolor
- Iris Virginica

Total Samples: **150**

Steps

1. Import required libraries.
2. Load the Iris dataset.
3. Split the dataset into training and testing sets.
4. Scale the feature values.
5. Build the ANN model.
6. Train the model.
7. Evaluate the model.
8. Predict the flower species.

Expected Outcome

The ANN model learns from the Iris dataset and predicts the flower species with high accuracy (approximately 95%–100%).

Result

The Artificial Neural Network (ANN) was successfully implemented using the Iris dataset, and the model classified flower species with high accuracy.

In [1]:
```
# Import Libraries
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

In [2]:
```
# Load Dataset

data = pd.read_csv(r"C:\Users\Smit\OneDrive\Desktop\Dataset\IRIS.csv")
print(data.head())
```

```
   sepal_length  sepal_width  petal_length  petal_width      species
0           5.1          3.5           1.4          0.2  Iris-setosa
1           4.9          3.0           1.4          0.2  Iris-setosa
2           4.7          3.2           1.3          0.2  Iris-setosa
3           4.6          3.1           1.5          0.2  Iris-setosa
4           5.0          3.6           1.4          0.2  Iris-setosa
```

In [3]:
```
# Features and Target

X = data.iloc[:, :-1]
y = data.iloc[:, -1]
```

In [4]:
```
# Encode Target

encoder = LabelEncoder()
y = encoder.fit_transform(y)
```

In [5]:
```
# Train-Test Split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

In [6]:
```
# Feature Scaling

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [7]:
```
# Build ANN Model

model = Sequential()
model.add(Dense(8, activation='relu', input_shape=(4,)))
model.add(Dense(8, activation='relu'))
model.add(Dense(3, activation='softmax'))
```

```
C:\Users\Smit\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:87:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. W
hen using Sequential models, prefer using an `Input(shape)` object as the fi
rst layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [8]:
```
# Compile Model

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

In [9]:
```
# Train Model

model.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=5,
    verbose=1
)
```

```
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9333 - loss: 0.1210
Epoch 58/100
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9288 - loss: 0.1024
Epoch 59/100
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9240 - loss: 0.1185
Epoch 60/100
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9593 - loss: 0.0802
Epoch 61/100
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9400 - loss: 0.1034
Epoch 62/100
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9533 - loss: 0.0965
Epoch 63/100
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9816 - loss: 0.0720
Epoch 64/100
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9467 - loss: 0.0881
Epoch 65/100
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9789 - loss: 0.0704
Epoch 66/100
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9596 - loss: 0.0751
Epoch 67/100
```

In [10]:
```
# Evaluate Model

loss, accuracy = model.evaluate(X_test, y_test)
print("\nAccuracy :", accuracy)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 367ms/step - accuracy: 0.9667 - loss: 0.0527

Accuracy : 0.9666666388511658
```

In [11]:
```
# Predict

predictions = model.predict(X_test)
predicted_classes = np.argmax(predictions, axis=1)

print("\nPredicted Classes")
print(predicted_classes)

print("\nActual Classes")
print(y_test)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 123ms/step

Predicted Classes
[1 0 2 1 1 0 1 2 2 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]

Actual Classes
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
```